Taller práctico: inicio de sesión con Firebase Authentication en React Native

Tecnología Superior en Desarrollo de Software, ESFOT

1. Propósito del taller

En este taller vamos a construir, paso a paso, una aplicación móvil con React Native y Expo que permite a un estudiante crear una cuenta, iniciar sesión, recuperar su contraseña por correo y cambiarla desde la propia app. Todo el manejo de usuarios lo resuelve Firebase Authentication, así que no necesitamos programar un servidor propio.

Al terminar, cada estudiante tendrá la app funcionando en su teléfono. Cuando el inicio de sesión es correcto, la app muestra el mensaje "Bienvenido a la APP, estudiante de la ESFOT", junto con los datos de la cuenta y las opciones para cambiar la contraseña o cerrar sesión.



Inicio de sesión



Bienvenida



Cambio de contraseña

Además del resultado visible, el taller trabaja cuatro ideas que se repiten en cualquier app profesional: separar la lógica de autenticación de las pantallas, proteger rutas según el estado de la sesión, validar datos antes de enviarlos y traducir los errores técnicos a mensajes que el usuario entienda.

1.1. Decisiones técnicas y por qué

- **Expo con Expo Router.** Expo evita instalar Android Studio o Xcode para empezar: el estudiante prueba la app en su teléfono con Expo Go. Expo Router organiza la navegación por archivos, igual que los frameworks web actuales.
- **SDK web de Firebase y no React Native Firebase.** Usamos el paquete `firebase` y no `@react-native-firebase`. El SDK web es JavaScript puro y funciona dentro de Expo Go. La librería nativa es más completa, pero obliga a compilar una versión propia de la app (development build).

- **Una sola capa que habla con Firebase.** Todas las llamadas a Firebase viven en el archivo `src/context/sesion.tsx`. Las pantallas solo llaman a funciones como `iniciarSesion` o `cambiarClave`.
- **TypeScript.** La plantilla de Expo ya viene con TypeScript y nos avisa de errores antes de ejecutar la app.

1.2. Limitaciones

- **El registro es abierto.** Cualquier persona con un correo puede crear una cuenta. El texto "estudiante de la ESFOT" es un saludo, no una verificación. Para restringir el acceso de verdad habría que crear las cuentas desde la consola o validar el dominio en un servidor.
- **El enlace de recuperación abre una página web de Firebase, no la app.**

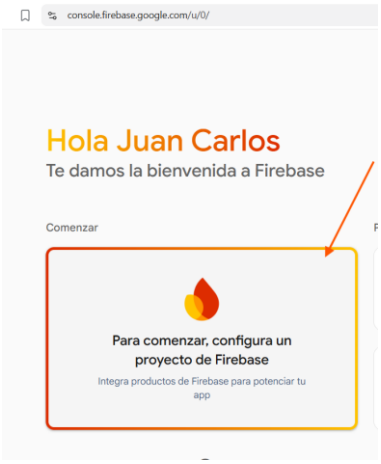
2. Requisitos previos

- Una **cuenta de Google** para entrar a la consola de Firebase.

3. Configuración del proyecto en Firebase

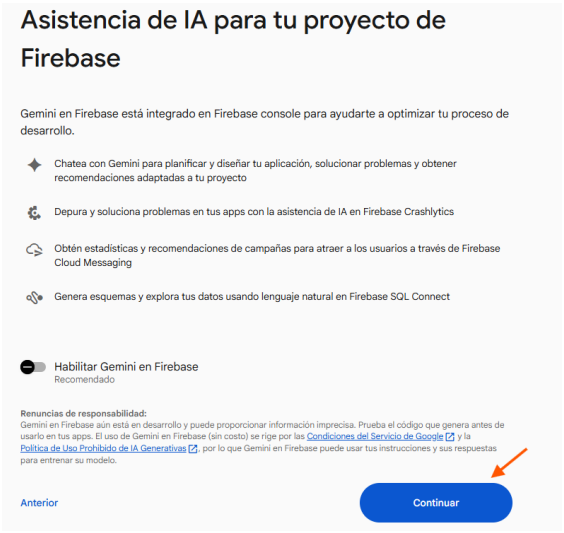
3.1. Crear el proyecto

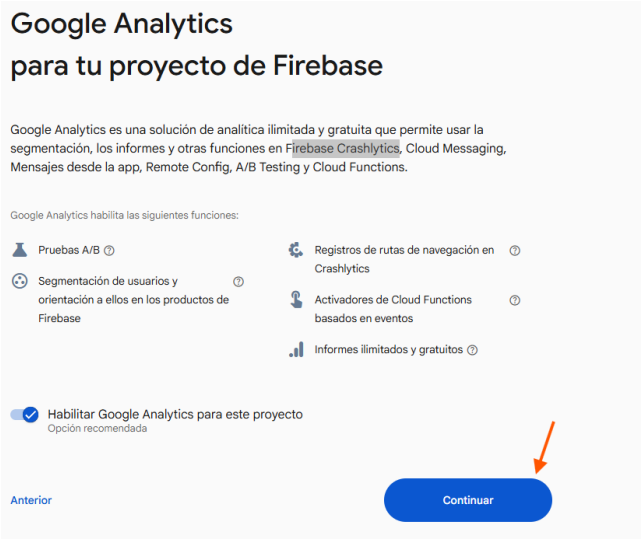
1) Entrar a `https://console.firebase.google.com` y elegir **Configurar un nuevo proyecto**.

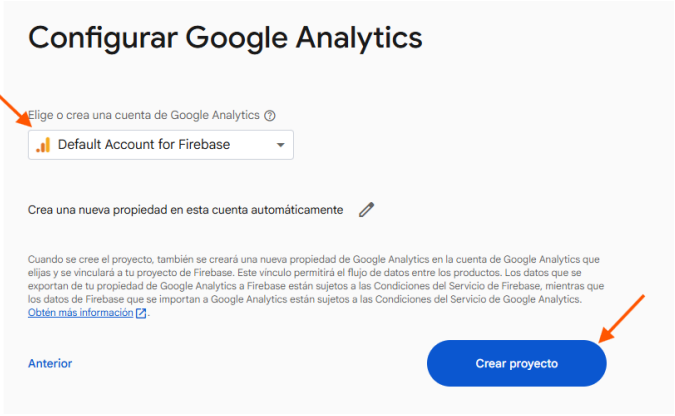


2) Ponerle un nombre, por ejemplo `esfot-login`. No es necesario dejar activada la incorporación con Gemini. Dejar activada la opción de Google Analytics que habilita luego Firebase Crashlytics

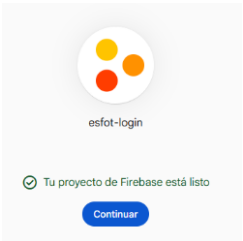








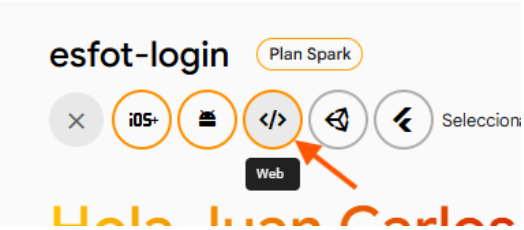
3) Esperar a que termine la creación y abrir el proyecto.



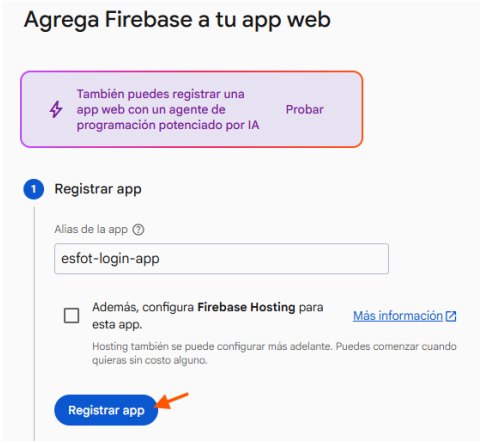
3.2. Registrar una app web y copiar la configuración

Aunque la app es móvil, la registramos como **app web**, porque usamos el SDK web de Firebase.

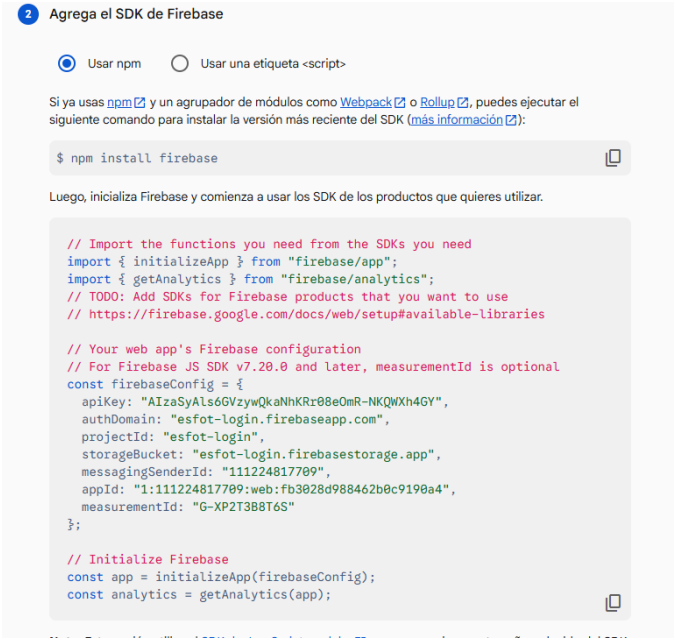
1) En la página principal del proyecto, tocar el ícono `</>` (Web).



2) Escribir un apodo, por ejemplo `esfot-login-app`. No hace falta configurar Firebase Hosting.



3) Firebase muestra un objeto `firebaseConfig` con seis valores: `apiKey`, `authDomain`, `projectId`, `storageBucket`, `messagingSenderId` , `appId`, `measurementId`. Copiarlos.

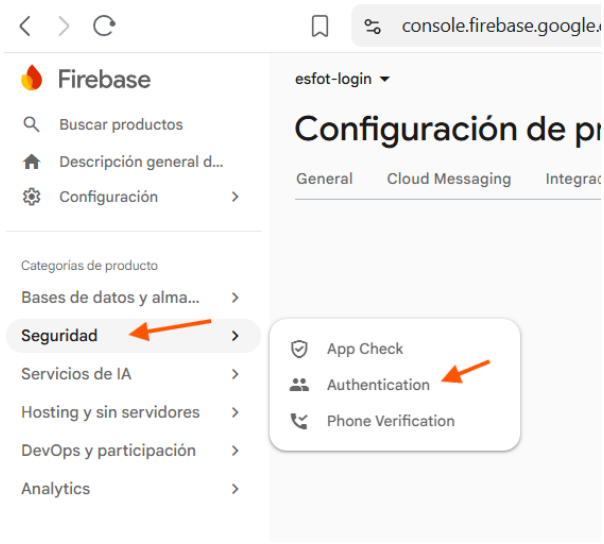


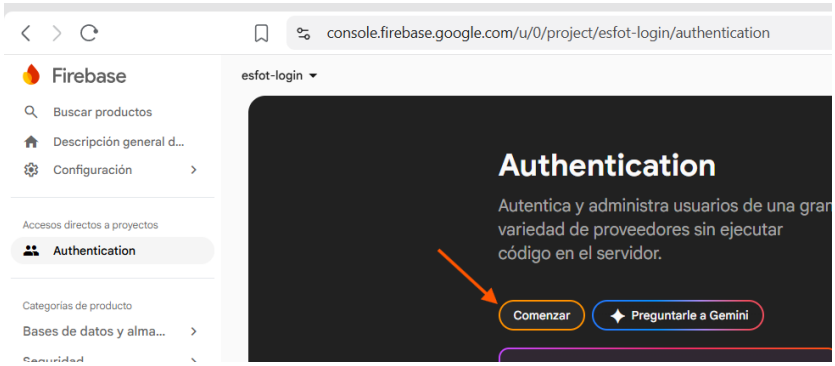


Estos valores no son secretos: viajan dentro de la app y cualquiera podría leerlos. La seguridad la ponen las reglas y la configuración de la consola. Aun así los sacamos del código para no subirlos al repositorio y para cambiar de proyecto sin tocar el código.

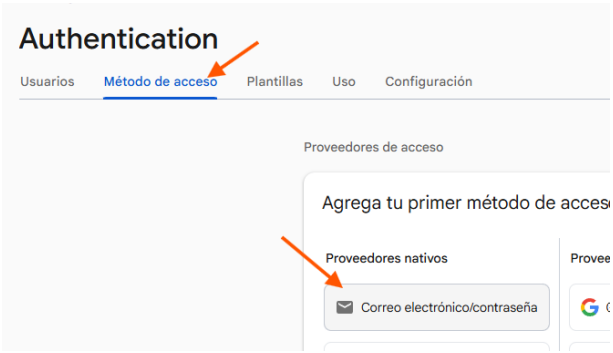
3.3. Habilitar el acceso con correo y contraseña

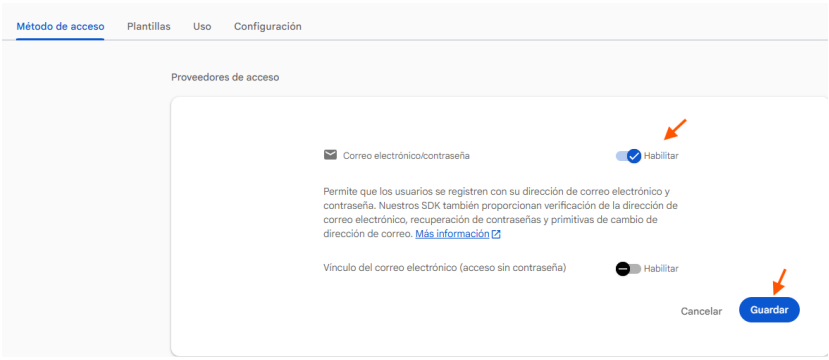
1) Menú lateral: **Seguridad > Authentication > Comenzar**.

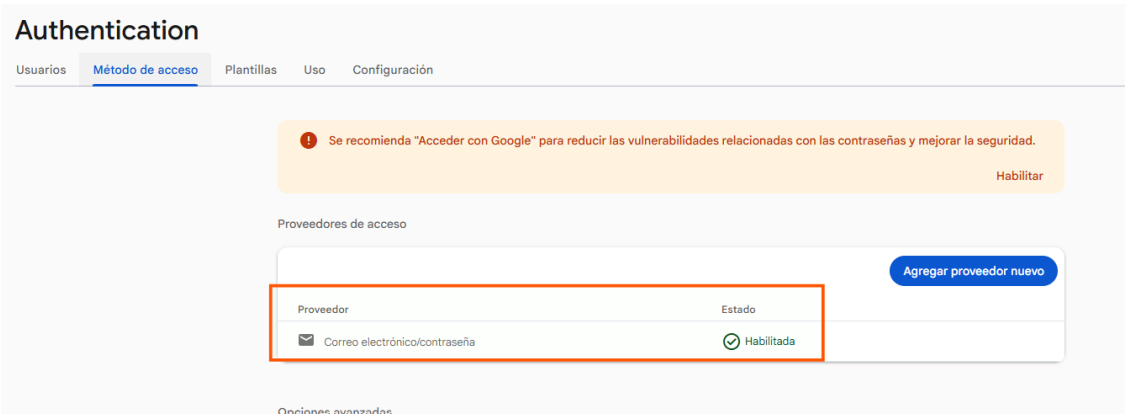




2) Pestaña **Método de acceso**: elegir **Correo electrónico/contraseña**, activar el primer interruptor y guardar. El segundo interruptor (vínculo de correo sin contraseña) se deja apagado.



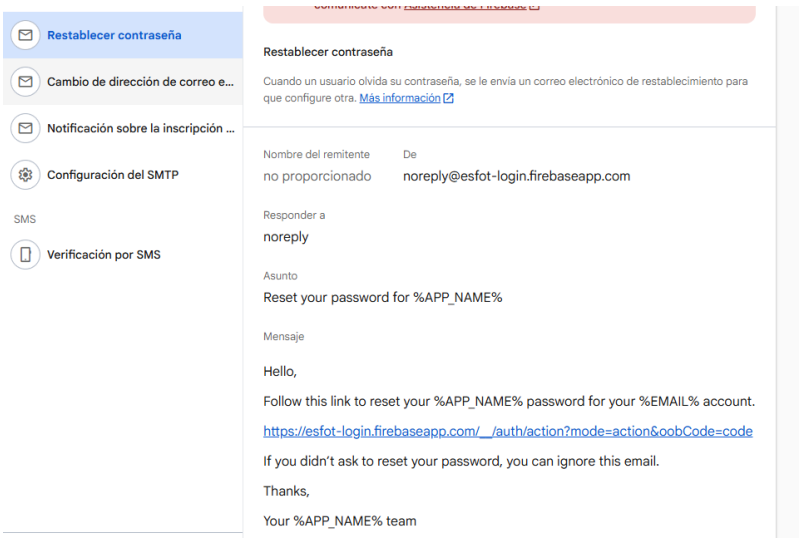




Si se omite este paso, la app responde con el error `auth/operation-not-allowed`. Es el error más común por omitir este paso.

3.4. Revisar la plantilla del correo de recuperación

1) En **Authentication > Plantillas**, abrir **Restablecimiento de contraseña**.
2) Revisar el nombre del remitente y el asunto. El idioma lo fija la app con `auth.languageCode = "es"` y también se puede cambiar en la consola con la opción de idioma de la plantilla.
3) El correo suele en ocasiones llegar a la carpeta de spam, porque sale de un dominio de Firebase. En un proyecto real se configura un dominio propio en esa misma pantalla.



4. Creación del proyecto Expo

4.1. Generar el proyecto

En una terminal, crear el proyecto:

```
npx create-expo-app@latest Login-Firebase
```

4.2. Limpiar la plantilla de ejemplo

La plantilla trae pantallas de demostración. Las eliminamos con el script incluido:

```
PS C:\Proyectos\Login-Firebase> npm run reset-project

> login-firebase@1.0.0 reset-project
> node ./scripts/reset-project.js

Do you want to move existing files to /example instead of deleting them? (Y/n): n
```

El script pregunta si queremos mover el ejemplo a otra carpeta. Respondemos n para borrarlo.

```
Do you want to move existing files to /example instead of deleting them? (Y/n): n
❌ /src deleted.
❌ /scripts deleted.

📁 New /src/app directory created.
📄 src/app/index.tsx created.
📄 src/app/_layout.tsx created.

✅ Project reset complete. Next steps:
1. Run `npx expo start` to start a development server.
2. Edit src/app/index.tsx to edit the main screen.
3. Put all your application code in /src, only screens and layout files should be in /src/app.
```

Después borramos también el archivo `src/app/index.tsx`.

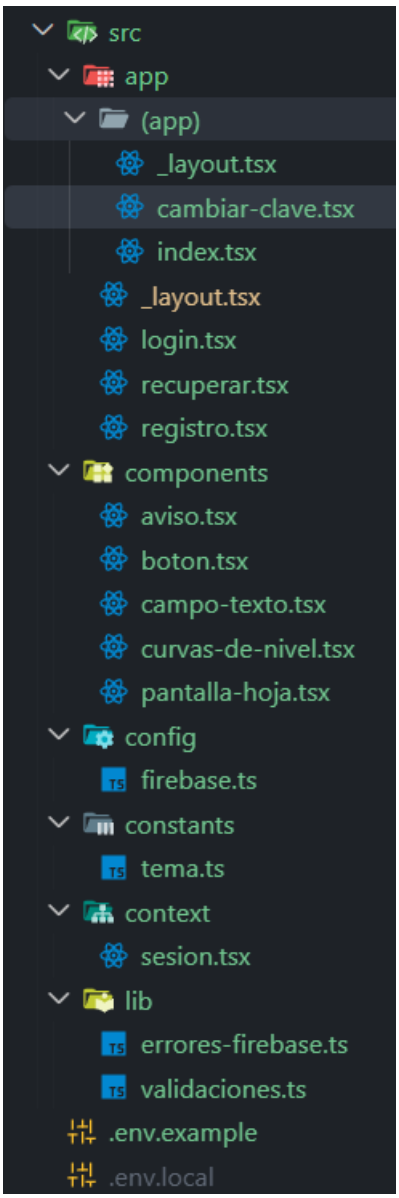
4.3. Instalar las dependencias

Paquete	Para qué sirve
`firebase`	SDK de Firebase: inicio de sesión, registro, recuperación y cambio de contraseña.
`@react-native-async-storage/async-storage`	Almacenamiento local del teléfono. Firebase guarda ahí la sesión para que no se pierda al cerrar la app.
`react-native-svg`	Dibujo vectorial: las curvas de nivel del encabezado y la flecha de volver.
`@expo-google-fonts/big-shoulders`	Fuente condensada para los títulos grandes.
`@expo-google-fonts/space-grotesk`	Fuente del texto, etiquetas y botones.

4.4. Estructura de carpetas

Expo Router trata cada archivo dentro de `src/app` como una pantalla. Todo lo que no es pantalla (componentes, lógica, configuración) va fuera de esa carpeta. Al final del taller el proyecto queda así:

```
src/
  app/
    _layout.tsx          fuentes, sesión y rutas protegidas
    login.tsx            inicio de sesión
    registro.tsx         crear cuenta
    recuperar.tsx        recuperar contraseña
    (app)/               solo con sesión iniciada
      _layout.tsx
      index.tsx          bienvenida
      cambiar-clave.tsx  cambiar contraseña
  components/
    aviso.tsx            mensajes de error y éxito
    boton.tsx            botón con tres variantes
    campo-texto.tsx      campo con etiqueta y error
    curvas-de-nivel.tsx  fondo topográfico
    pantalla-hoja.tsx    plantilla común de pantallas
  config/
    firebase.ts          conexión con Firebase (móvil)
  constants/
    tema.ts              colores, fuentes y medidas
  context/
    sesion.tsx           toda la lógica de autenticación
  lib/
    errores-firebase.ts  mensajes de error en español
    validaciones.ts      validaciones de formularios
```



La carpeta `(app)` lleva paréntesis porque es un **grupo**: sirve para organizar y aplicar un layout común, pero no aparece en la ruta. La bienvenida es `/` y no `/app`.

5. Variables de entorno

En la raíz del proyecto creamos el archivo `.env.example`, que sí se sube al repositorio como referencia, y una copia llamada `.env.local` con los valores reales copiados de la consola en la sección 3.2.

Archivo **`.env.example`**

```
# Copia este archivo como ".env.local" y reemplaza los valores con los de tu proyecto.
# El prefijo EXPO_PUBLIC_ es obligatorio para que Expo inyecte la variable en la app.
EXPO_PUBLIC_FIREBASE_API_KEY=tu-api-key
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=tu-proyecto.firebaseapp.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=tu-proyecto
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=tu-proyecto.firebasestorage.app
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=000000000000
EXPO_PUBLIC_FIREBASE_APP_ID=1:000000000000:web:00000000000000000000
```

Archivo **.env.example(pondrán sus datos, no los míos, que he de creer si se jalan)**

```
.env.local
EXPO_PUBLIC_FIREBASE_API_KEY=AIzaSyAls6GVzywQkaNhKRr08eOmR-NKQWXh4GY
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=esfot-login.firebaseapp.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=esfot-login
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=esfot-login.firebasestorage.app
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=111224817709
EXPO_PUBLIC_FIREBASE_APP_ID=1:111224817709:web:fb3028d988462b0c9190a4
```

Expo solo expone a la app las variables cuyo nombre empieza con `EXPO_PUBLIC_`. El archivo `.gitignore` de la plantilla ya excluye `.env*.local`, así que los valores reales no llegan a GitHub. Si se modifica `.env.local` con el servidor encendido, hay que reiniciarlo con `npx expo start -c`.

6. Conexión con Firebase

6.1. Ajuste de TypeScript para la persistencia de sesión

Para que la sesión sobreviva al cierre de la app usamos la función `getReactNativePersistence`. La función existe cuando la app corre en el teléfono, pero las definiciones de tipos que TypeScript lee por defecto son las del navegador, donde no existe. El resultado es un error falso: "Module firebase/auth has no exported member getReactNativePersistence".

La solución es indicarle a TypeScript que lea los tipos de la versión React Native de Firebase. En `tsconfig.json` agregamos una entrada en `paths`:

Archivo **`tsconfig.json`**

```
tsconfig.json > ...
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "paths": {
      "@/*": [
        "./src/*"
      ],
      "@/assets/*": [
        "./assets/*"
      ],
      "@firebase/auth": [
        "./node_modules/@firebase/auth/dist/index.rn.d.ts"
      ]
    }
  },
  "include": [
    "**/*.ts",
    "**/*.tsx",
    ".expo/types/**/*.ts",
    "expo-env.d.ts"
  ]
}
```

Este ajuste solo afecta al chequeo de tipos. El empaquetador de Expo (Metro) ya elige por su cuenta la versión React Native de Firebase al compilar para el teléfono.

6.2. Archivo de conexión para el móvil

Archivo **`src/config/firebase.ts`**

```
/**
 * Conexión con Firebase para Android e iOS.
 * Expo elige este archivo en el móvil.
 */
import AsyncStorage from '@react-native-async-storage/async-storage';
import { getApp, getApps, initializeApp } from 'firebase/app';
```

```ts
import { getAuth, getReactNativePersistence, initializeAuth, type Auth } from
'firebase/auth';

// Los valores vienen del archivo .env.local. Expo solo expone a la app
// las variables que empiezan con EXPO_PUBLIC_.
const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};

// Con la recarga rapida (Fast Refresh) este módulo se puede ejecutar más de
// una vez. getApps() evita el error "Firebase App named '[DEFAULT]' already
exists".
const app = getApps().length ? getApp() : initializeApp(firebaseConfig);

// initializeAuth solo se puede llamar una vez por app. Si ya existe, lo
reutilizamos.
function crearAuth(): Auth {
  try {
    // getReactNativePersistence guarda la sesión en AsyncStorage: así el
    // estudiante sigue conectado aunque cierre la app. Sin esto, la sesión
    // vive solo en memoria y se pierde al reiniciar.
    return initializeAuth(app, {
      persistence: getReactNativePersistence(AsyncStorage),
    });
  } catch {
    return getAuth(app);
  }
}

export const auth = crearAuth();

// Los correos de recuperacion de contraseña llegarán en español.
auth.languageCode = 'es';
```

- `getApps()` evita inicializar Firebase dos veces cuando Expo recarga el código en caliente.
- `initializeAuth` solo se puede llamar una vez; por eso está dentro de un `try` y, si falla, se reutiliza la instancia existente con `getAuth`.
- Sin la persistencia de AsyncStorage la app funcionaría igual, pero el usuario tendría que iniciar sesión cada vez que la abre.

7. Tema visual.

Todos los colores, fuentes y medidas viven en un solo archivo. Si mañana cambia el color principal, se cambia aquí y toda la app se actualiza.

Archivo **`src/constants/tema.ts`**

```ts
/**
 * Tema visual de la app.
 *
 * Centralizar colores, tipografías y medidas en un solo archivo evita
 * "números mágicos" regados por las pantallas: si mañana cambia el color
 * principal, se cambia aquí y toda la app se actualiza.
 *
*/

export const colores = {
  tinta: '#14213D', // azul politécnico: fondo del encabezado
  tintaProfunda: '#0C1529', // sombra del encabezado y texto sobre amarillo
  curva: '#2C3D68', // curvas de nivel dibujadas sobre la tinta
  papel: '#EEF1F5', // fondo de la hoja (gris frío, no crema)
  hoja: '#FFFFFF', // superficie de los campos y tarjetas
  senal: '#F5B700', // amarillo de seguridad: botón principal
  grafito: '#1F232B', // texto principal sobre la hoja
  lapiz: '#5A6274', // texto secundario y etiquetas
  borde: '#CBD2DD', // bordes de campos en reposo
  error: '#B3361F',
  errorFondo: '#FBE9E5',
  exito: '#22704F',
  exitoFondo: '#E3F3EA',
} as const;

/**
 * Nombres de las fuentes tal como se registran en useFonts (ver _layout.tsx).
 * Big Shoulders es una condensada de rotulación industrial: da carácter a
 * los titulares. Space Grotesk es legible y técnica para el resto del texto.
 */
export const fuentes = {
  titular: 'BigShoulders_800ExtraBold',
  texto: 'SpaceGrotesk_400Regular',
  textoMedio: 'SpaceGrotesk_500Medium',
  textoFuerte: 'SpaceGrotesk_700Bold',
} as const;

/** Escala de espacios basada en múltiplos de 4 px. */
export const espacio = {
  xs: 4,
  s: 8,
  m: 16,
  l: 24,
  xl: 32,
  xxl: 48,
} as const;

/** Radios: la hoja es más redondeada que los controles para marcar jerarquía. */
export const radio = {
  control: 12,
  hoja: 28,
} as const;
```

8. Validaciones y mensajes de error

8.1. Validaciones del formulario

Validar en el teléfono antes de llamar a Firebase da respuesta inmediata y ahorra peticiones. No reemplaza la validación del servidor: Firebase vuelve a revisar todo. Cada función devuelve un mensaje de error o `null` si el valor es válido, lo que permite mostrar el mensaje directamente debajo del campo.

Archivo **`src/lib/validaciones.ts`**

```ts
/**
 * Validaciones del lado del cliente.
 * Sirven para dar respuesta inmediata sin gastar una petición a Firebase.
 * No reemplazan la validación del servidor: Firebase vuelve a validar todo.
 * Cada función devuelve un mensaje de error o null si el valor es válido.
 */

// Patrón simple: algo@algo.dominio. La validación estricta la hace Firebase.
const PATRON_CORREO = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;

/** Firebase exige 6 caracteres como mínimo; en el taller pedimos 8. */
export const LONGITUD_MINIMA_CLAVE = 8;

export function validarCorreo(correo: string): string | null {
  if (!correo.trim()) return 'Escribe tu correo.';
  if (!PATRON_CORREO.test(correo.trim())) return 'Revisa el formato del correo.';
  return null;
}

export function validarClave(clave: string): string | null {
  if (!clave) return 'Escribe una contraseña.';
  if (clave.length < LONGITUD_MINIMA_CLAVE) {
    return `Usa al menos ${LONGITUD_MINIMA_CLAVE} caracteres.`;
  }
  return null;
}

export function validarConfirmacion(clave: string, confirmacion: string): string | null {
  if (!confirmacion) return 'Repite la contraseña.';
  if (clave !== confirmacion) return 'Las contraseñas no coinciden.';
  return null;
}

export function validarNombre(nombre: string): string | null {
  if (nombre.trim().length < 3) return 'Escribe tu nombre (mínimo 3 letras).';
  return null;
}
```

Firebase acepta contraseñas desde 6 caracteres. Pedimos 8 como punto de partida para el taller, sin embargo la guía NIST SP 800-63B-4(Directrices de identidad digital) pide al menos 15 caracteres cuando la contraseña es el único factor de autenticación, y 8 cuando se combina con un segundo factor. El valor está en una constante, así que cambiarlo es una línea.

8.2. Traducción de los errores de Firebase

Firebase devuelve códigos como `auth/invalid-credential`. Mostrar eso al usuario no sirve: el mensaje debe decir qué pasó y cómo corregirlo. Este archivo hace la traducción y permite que cada pantalla reemplace un mensaje cuando el mismo código significa algo distinto en su contexto.

Archivo **`src/lib/errores-firebase.ts`**

```
/**
 * Traduce los códigos de error de Firebase Auth a mensajes claros en español.
 *
 * Firebase devuelve errores como "auth/invalid-credential". Mostrar eso al
 * usuario no sirve: el mensaje debe decir que paso y cómo corregirlo.
 */
import { FirebaseError } from 'firebase/app';

const mensajes: Record<string, string> = {
  // Desde 2023 los proyectos nuevos tienen activada la "protección contra la
  // enumeración de correos": Firebase ya no distingue entre "el correo no
  // existe" y "la contraseña es incorrecta". Ambos llegan como invalid-credential.
  'auth/invalid-credential': 'El correo o la contraseña no coinciden. Revisa los datos e inténtalo
de nuevo.',
  'auth/wrong-password': 'El correo o la contraseña no coinciden. Revisa los datos e inténtalo de
nuevo.',
  'auth/user-not-found': 'El correo o la contraseña no coinciden. Revisa los datos e inténtalo de
nuevo.',
  'auth/invalid-email': 'El correo no tiene un formato válido. Ejemplo:
nombre.apellido@epn.edu.ec',
  'auth/missing-email': 'Escribe tu correo para continuar.',
  'auth/missing-password': 'Escribe tu contraseña para continuar.',
  'auth/email-already-in-use': 'Ya existe una cuenta con ese correo. Inicia sesión o recupera tu
contraseña.',
  'auth/weak-password': 'La contraseña es muy débil. Usa al menos 8 caracteres.',
  'auth/password-does-not-meet-requirements':
    'La contraseña no cumple la política del proyecto. Combina mayúsculas, minúsculas y números.',
  'auth/too-many-requests': 'Hubo demasiados intentos seguidos. Espera unos minutos y vuelve a
intentarlo.',
  'auth/network-request-failed': 'No hay conexión con Firebase. Revisa tu internet e inténtalo
otra vez.',
  'auth/requires-recent-login': 'Por seguridad, cierra sesión, vuelve a ingresar y repite el
cambio.',
  'auth/user-disabled': 'Esta cuenta fue deshabilitada. Comunícate con el administrador.',
  // Este suele aparecer en clase: falta habilitar el proveedor en la consola.
  'auth/operation-not-allowed':
    'El acceso con correo y contraseña no está habilitado en la consola de Firebase.',
```

```
  'auth/invalid-api-key': 'La configuración de Firebase no es válida. Revisa el archivo
.env.local.',
};

/**
 * @param error  Lo que llegó al bloque catch.
 * @param reemplazos  Mensajes propios de una pantalla para ciertos códigos.
 *   Ejemplo: al cambiar la contraseña, invalid-credential significa
 *   "la contraseña actual no es correcta", no "el correo no coincide".
 */
export function mensajeDeError(error: unknown, reemplazos: Record<string, string> = {}): string {
  if (error instanceof FirebaseError) {
    if (reemplazos[error.code]) return reemplazos[error.code];
    if (mensajes[error.code]) return mensajes[error.code];
    // La API key mal copiada produce un código largo que empieza así.
    if (error.code.startsWith('auth/api-key-not-valid')) return mensajes['auth/invalid-api-key'];
    // Para errores no previstos mostramos el código: ayuda a depurar en el taller.
    return `No se pudo completar la operación (${error.code}).`;
  }
  return 'Ocurrió un error inesperado. Inténtalo de nuevo.';
}
```

9. Contexto de sesión

Este archivo es el corazón del taller. Usa el Context API de React para que cualquier pantalla pueda saber quién está conectado y llamar a las operaciones de autenticación, sin pasar datos de componente en componente.

Archivo **`src/context/sesion.tsx`**

```
/**
 * Contexto de sesión: el único lugar de la app que habla con Firebase Auth.
 *
 * Las pantallas no importan Firebase directamente; llaman a las funciones de
 * este contexto (iniciarSesion, registrarse, etc.). Así, si mañana se cambia
 * Firebase por otro proveedor, solo se reescribe este archivo.
 */
import {
  createUserWithEmailAndPassword,
  EmailAuthProvider,
  onAuthStateChanged,
  reauthenticateWithCredential,
  sendPasswordResetEmail,
  signInWithEmailAndPassword,
  signOut,
  updatePassword,
  updateProfile,
  type User,
} from 'firebase/auth';
import { createContext, use, useEffect, useState, type ReactNode } from 'react';

import { auth } from '@/config/firebase';

type ValorSesion = {
```

```tsx
  /** Usuario conectado, o null si nadie ha iniciado sesión. */
  usuario: User | null;
  /** Nombre del perfil (displayName), siempre actualizado tras el registro. */
  nombre: string | null;
  /** true mientras Firebase revisa si hay una sesión guardada. */
  cargando: boolean;
  iniciarSesion: (correo: string, clave: string) => Promise<void>;
  registrarse: (nombre: string, correo: string, clave: string) => Promise<void>;
  enviarCorreoRecuperacion: (correo: string) => Promise<void>;
  cambiarClave: (claveActual: string, claveNueva: string) => Promise<void>;
  cerrarSesion: () => Promise<void>;
};

const ContextoSesion = createContext<ValorSesion | null>(null);

/** Hook para leer la sesión desde cualquier pantalla. */
export function useSesion(): ValorSesion {
  // `use` es la forma moderna (React 19) de leer un contexto.
  const valor = use(ContextoSesion);
  if (!valor) {
    throw new Error('useSesion debe usarse dentro de <ProveedorSesion>.');
  }
  return valor;
}

export function ProveedorSesion({ children }: { children: ReactNode }) {
  const [usuario, setUsuario] = useState<User | null>(null);
  const [cargando, setCargando] = useState(true);
  // Nombre del perfil guardado aparte: updateProfile modifica el mismo objeto
  // `user` sin cambiar su referencia, así que React no detectaría el cambio
  // si las pantallas leyeran usuario.displayName directamente.
  const [nombre, setNombre] = useState<string | null>(null);

  useEffect(() => {
    // onAuthStateChanged se dispara al arrancar (con la sesión guardada o null)
    // y cada vez que alguien inicia o cierra sesión. Es la fuente de verdad:
    // la navegación (Stack.Protected en _layout.tsx) depende de este estado.
    const cancelarSuscripcion = onAuthStateChanged(auth, (usuarioFirebase) => {
      setUsuario(usuarioFirebase);
      setNombre(usuarioFirebase?.displayName ?? null);
      setCargando(false);
    });
    // Al desmontar el proveedor dejamos de escuchar para no fugar memoria.
    return cancelarSuscripcion;
  }, []);

  async function iniciarSesion(correo: string, clave: string) {
    await signInWithEmailAndPassword(auth, correo.trim(), clave);
    // No navegamos aquí: onAuthStateChanged actualiza `usuario` y el
    // Stack.Protected muestra automáticamente la pantalla de bienvenida.
  }

  async function registrarse(nombreCompleto: string, correo: string, clave: string) {
```

```
    // Crear la cuenta también inicia la sesión.
    const credencial = await createUserWithEmailAndPassword(auth, correo.trim(), clave);
    // Guardamos el nombre en el perfil de Firebase (campo displayName).
    const nombreLimpio = nombreCompleto.trim();
    await updateProfile(credencial.user, { displayName: nombreLimpio });
    // La bienvenida ya se mostró (onAuthStateChanged corre al crear la cuenta,
    // antes de updateProfile), así que actualizamos el nombre en el estado.
    setNombre(nombreLimpio);
  }

  async function enviarCorreoRecuperacion(correo: string) {
    // Firebase envía un correo con un enlace a su página de restablecimiento.
    // Si el correo no existe, por la protección de enumeración NO lanza error:
    // por eso la pantalla muestra un mensaje neutro.
    await sendPasswordResetEmail(auth, correo.trim());
  }

  async function cambiarClave(claveActual: string, claveNueva: string) {
    const actual = auth.currentUser;
    if (!actual?.email) throw new Error('No hay una sesión activa.');

    // updatePassword es una operación sensible: Firebase la rechaza con
    // "auth/requires-recent-login" si la sesión no es reciente. Por eso
    // primero confirmamos la identidad con la contraseña actual.
    const credencial = EmailAuthProvider.credential(actual.email, claveActual);
    await reauthenticateWithCredential(actual, credencial);
    await updatePassword(actual, claveNueva);
  }

  async function cerrarSesion() {
    await signOut(auth);
  }

  return (
    <ContextoSesion
      value={{
        usuario,
        nombre,
        cargando,
        iniciarSesion,
        registrarse,
        enviarCorreoRecuperacion,
        cambiarClave,
        cerrarSesion,
      }}>
      {children}
    </ContextoSesion>
  );
}
```



Puntos clave:

- **`onAuthStateChanged` es la fuente de verdad.** Se ejecuta al abrir la app (con la sesión guardada o con `null`) y cada vez que alguien entra o sale. Por eso `iniciarSesion` no navega a ninguna parte: cuando Firebase confirma el ingreso, el estado cambia y la navegación reacciona sola.
- **`cargando` empieza en `true`.** Mientras Firebase revisa si había una sesión guardada no sabemos qué pantalla mostrar. Si no esperamos, la app mostraría el login por un instante aunque el usuario ya estuviera conectado.
- **Cambiar la contraseña exige reautenticar.** `updatePassword` es una operación sensible y Firebase la rechaza con `auth/requires-recent-login` si el último ingreso no es reciente. Por eso primero creamos una credencial con la contraseña actual y llamamos a `reauthenticateWithCredential`.
- **El contador `versionPerfil`.** `updateProfile` modifica el mismo objeto de usuario y React no detecta el cambio. Subir un contador obliga a volver a dibujar la pantalla, y así la bienvenida muestra el nombre recién guardado.
- **`use(ContextoSesion)`** es la forma de leer un contexto en React 19. Es equivalente a `useContext`, que también funciona.

10. Componentes de interfaz

Antes de las pantallas construimos las piezas que se repiten. Así cada pantalla queda corta y se ocupa solo de su lógica.

10.1. Fondo de curvas de nivel

El fondo no es una imagen: las curvas se calculan con trigonometría y se dibujan con SVG. Pesan casi nada y se ven nítidas en cualquier pantalla. El diseño también se programa.

Archivo **`src/components/curvas-de-nivel.tsx`**

```tsx
/**
 * Fondo de curvas de nivel (como un plano topográfico del Pichincha).
 *
 * No es una imagen: las curvas se calculan con matemáticas y se dibujan con
 * react-native-svg. Así pesan casi nada y se ven nítidas en cualquier pantalla.
 *
 * Cómo se construye cada curva:
 *   1. Se toman 64 ángulos alrededor de una "cumbre" (el centro).
 *   2. Para cada ángulo se calcula un radio que ondula con senos: así la
 *      curva no es un círculo perfecto sino un contorno irregular de montaña.
 *   3. Los puntos se unen con curvas Bézier suaves (técnica Catmull-Rom).
 *   4. Se repite con radios cada vez más grandes: cada anillo es una "cota".
 */
import { useMemo } from 'react';
import { StyleSheet } from 'react-native';
import Svg, { Circle, Path } from 'react-native-svg';

import { colores } from '@/constants/tema';

// Tamaño del "lienzo" interno. El SVG se escala al tamaño real del contenedor.
const ANCHO = 400;
const ALTO = 560;

type Punto = { x: number; y: number };

/** Convierte una lista cerrada de puntos en un trazo SVG suave. */
function trazoSuave(puntos: Punto[]): string {
  const n = puntos.length;
  let d = `M ${puntos[0].x.toFixed(1)} ${puntos[0].y.toFixed(1)}`;
  for (let i = 0; i < n; i++) {
    // Para cada tramo usamos el punto anterior y el siguiente como guía.
    const p0 = puntos[(i - 1 + n) % n];
    const p1 = puntos[i];
    const p2 = puntos[(i + 1) % n];
    const p3 = puntos[(i + 2) % n];
    const c1 = { x: p1.x + (p2.x - p0.x) / 6, y: p1.y + (p2.y - p0.y) / 6 };
    const c2 = { x: p2.x - (p3.x - p1.x) / 6, y: p2.y - (p3.y - p1.y) / 6 };
    d += ` C ${c1.x.toFixed(1)} ${c1.y.toFixed(1)}, ${c2.x.toFixed(1)} ${c2.y.toFixed(1)}, ${p2.x.toFixed(1)}
${p2.y.toFixed(1)}`;
  }
  return `${d} Z`;
}

/** Genera los anillos alrededor de una cumbre ubicada en (cx, cy). */
function generarCurvas(cx: number, cy: number, anillos: number): string[] {
  const curvas: string[] = [];
  for (let k = 1; k <= anillos; k++) {
    const radioBase = 14 + k * 21;
```

```
    const puntos: Punto[] = [];
    for (let i = 0; i < 64; i++) {
      const angulo = (i / 64) * Math.PI * 2;
      // La ondulación crece con el anillo: las cotas bajas son más irregulares.
      const ondulacion =
        Math.sin(angulo * 3 + k * 0.55) * (4 + k * 1.6) +
        Math.sin(angulo * 5 - k * 0.8) * (2 + k * 0.7) +
        Math.cos(angulo * 2 + k * 0.3) * k * 1.3;
      const r = radioBase + ondulacion;
      // Estiramos en X para que la montaña sea alargada, como una cordillera.
      puntos.push({ x: cx + Math.cos(angulo) * r * 1.35, y: cy + Math.sin(angulo) * r });
    }
    curvas.push(trazoSuave(puntos));
  }
  return curvas;
}

export function CurvasDeNivel() {
  // useMemo: los trazos se calculan una sola vez y no en cada render.
  const curvas = useMemo(() => generarCurvas(300, 70, 20), []);
  const cumbre = { x: 300, y: 70 };

  return (
    <Svg
      style={StyleSheet.absoluteFill}
      viewBox={`0 0 ${ANCHO} ${ALTO}`}
      // "slice" recorta como object-fit: cover, sin deformar las curvas.
      // "YMin" ancla arriba: la cumbre queda siempre a la vista.
      preserveAspectRatio="xMidYMin slice"
      pointerEvents="none">
      {curvas.map((d, i) => (
        <Path
          key={i}
          d={d}
          fill="none"
          stroke={colores.curva}
          // Cada quinta curva es una "curva maestra", más gruesa, como en los mapas reales.
          strokeWidth={i % 5 === 4 ? 1.8 : 0.9}
        />
      ))}
      {/* Punto de cota en la cumbre: el único detalle amarillo del encabezado. */}
      <Circle cx={cumbre.x} cy={cumbre.y} r={3.5} fill={colores.senal} />
      <Circle cx={cumbre.x} cy={cumbre.y} r={9} fill="none" stroke={colores.senal} strokeWidth={1} />
    </Svg>
  );
}
```

10.2. Plantilla común de pantalla

Todas las pantallas comparten la misma estructura: encabezado oscuro con el título y la hoja clara con el contenido. Este componente resuelve además dos problemas típicos de los formularios móviles: el teclado que tapa los campos y la muesca de la cámara.

Archivo **`src/components/pantalla-hoja.tsx`**

```tsx
/**
 * Plantilla común de todas las pantallas:
 *
 * Centralizar esto en un componente hace que cada pantalla solo se ocupe
 * de su lógica y no repita el manejo del teclado ni de las zonas seguras.
 */
import { router } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import type { ReactNode } from 'react';
import {
  KeyboardAvoidingView,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import Svg, { Path } from 'react-native-svg';

import { CurvasDeNivel } from '@/components/curvas-de-nivel';
import { colores, espacio, fuentes, radio } from '@/constants/tema';

type Props = {
  /** Línea pequeña sobre el título (por ejemplo, el nombre de la escuela). */
  antetitulo?: string;
  titulo: string;
  bajada?: string;
  /** Muestra el botón "Volver" en el encabezado. */
  conVolver?: boolean;
  /** Contenido extra dentro del encabezado, debajo del título. */
  extraEncabezado?: ReactNode;
  children: ReactNode;
};

export function PantallaHoja({
  antetitulo,
  titulo,
  bajada,
  conVolver = false,
  extraEncabezado,
  children,
}: Props) {
  // insets = espacio ocupado por la muesca (notch) y la barra de gestos.
  const insets = useSafeAreaInsets();

  return (
    // El teclado tapa los campos; KeyboardAvoidingView empuja el contenido.
    // También se necesita en Android: con edge-to-edge (siempre activo en SDK
    // recientes) el sistema ya no redimensiona la ventana al abrir el teclado.
    <KeyboardAvoidingView style={styles.raiz} behavior="padding">
      {/* Íconos de la barra de estado en blanco, porque el fondo es oscuro. */}
```

```
      <StatusBar style="light" />
      <ScrollView
        contentContainerStyle={styles.scroll}
        // "handled": un toque en un botón funciona aunque el teclado esté abierto.
        keyboardShouldPersistTaps="handled"
        bounces={false}>
        <View style={[styles.encabezado, { paddingTop: insets.top + espacio.m }]}>
          <CurvasDeNivel />

          {conVolver && (
            <Pressable
              onPress={() => router.back()}
              accessibilityRole="button"
              accessibilityLabel="Volver"
              hitSlop={12}
              style={({ pressed }) => [
                styles.volver,
                { top: insets.top + espacio.s },
                pressed && { opacity: 0.6 },
              ]}>
              <Svg width={18} height={18} viewBox="0 0 24 24">
                <Path
                  d="M15 5 L8 12 L15 19"
                  stroke={colores.papel}
                  strokeWidth={2.4}
                  fill="none"
                  strokeLinecap="round"
                  strokeLinejoin="round"
                />
              </Svg>
              <Text style={styles.volverTexto}>Volver</Text>
            </Pressable>
          )}

          <View style={styles.bloqueTitulo}>
            {antetitulo ? <Text style={styles.antetitulo}>{antetitulo}</Text> : null}
            {/* accessibilityRole="header" ayuda a los lectores de pantalla. */}
            <Text style={styles.titulo} accessibilityRole="header">
              {titulo}
            </Text>
            {bajada ? <Text style={styles.bajada}>{bajada}</Text> : null}
            {extraEncabezado}
          </View>
        </View>

        <View style={[styles.hoja, { paddingBottom: insets.bottom + espacio.xl }]}>
          {children}
        </View>
      </ScrollView>
    </KeyboardAvoidingView>
  );
}
```

```
const styles = StyleSheet.create({
  raiz: {
    flex: 1,
    backgroundColor: colores.tinta,
  },
  scroll: {
    // flexGrow permite que la hoja ocupe todo el alto aunque haya poco contenido.
    flexGrow: 1,
  },
  encabezado: {
    backgroundColor: colores.tinta,
    paddingHorizontal: espacio.l,
    // El espacio extra abajo queda oculto bajo la hoja (que sube con margen negativo).
    paddingBottom: espacio.xxl + radio.hoja,
    overflow: 'hidden',
    minHeight: 280,
    justifyContent: 'flex-end',
  },
  volver: {
    position: 'absolute',
    left: espacio.m,
    flexDirection: 'row',
    alignItems: 'center',
    gap: 2,
    paddingVertical: espacio.s,
    paddingHorizontal: espacio.s,
    // top se calcula en el render con insets.top (depende del teléfono).
  },
  volverTexto: {
    color: colores.papel,
    fontFamily: fuentes.textoMedio,
    fontSize: 15,
  },
  bloqueTitulo: {
    marginTop: espacio.xxl + espacio.l,
    maxWidth: 520,
  },
  antetitulo: {
    color: colores.senal,
    fontFamily: fuentes.textoMedio,
    fontSize: 14,
    letterSpacing: 0.2,
    marginBottom: espacio.s,
  },
  titulo: {
    color: colores.papel,
    fontFamily: fuentes.titular,
    fontSize: 58,
    lineHeight: 56,
    letterSpacing: -0.5,
  },
  bajada: {
    color: colores.papel,
```

```
    opacity: 0.78,
    fontFamily: fuentes.texto,
    fontSize: 16,
    lineHeight: 23,
    marginTop: espacio.m,
    maxWidth: 340,
  },
  hoja: {
    flex: 1,
    backgroundColor: colores.papel,
    borderTopLeftRadius: radio.hoja,
    borderTopRightRadius: radio.hoja,
    // La hoja "sube" sobre el encabezado para que parezca apoyada encima.
    marginTop: -radio.hoja,
    paddingHorizontal: espacio.l,
    paddingTop: espacio.xl,
    gap: espacio.m,
  },
});
```

10.3. Campo de texto

Archivo **`src/components/campo-texto.tsx`**

```
/**
 * Campo de texto con etiqueta, estado de foco, mensaje de error y,
 * si es una contraseña, el botón para mostrarla u ocultarla.
 */
import { useState } from 'react';
import { Pressable, StyleSheet, Text, TextInput, View, type TextInputProps } from 'react-native';

import { colores, espacio, fuentes, radio } from '@/constants/tema';

type Props = TextInputProps & {
  etiqueta: string;
  error?: string | null;
  /** Si es true, el texto se oculta y aparece el botón Mostrar/Ocultar. */
  esClave?: boolean;
};

export function CampoTexto({ etiqueta, error, esClave = false, style, ...props }: Props) {
  const [enfocado, setEnfocado] = useState(false);
  const [claveVisible, setClaveVisible] = useState(false);

  // El color del borde comunica el estado: error > foco > reposo.
  const colorBorde = error ? colores.error : enfocado ? colores.tinta : colores.borde;

  return (
    <View style={styles.contenedor}>
      <Text style={styles.etiqueta}>{etiqueta}</Text>

      <View style={[styles.caja, { borderColor: colorBorde }, enfocado && styles.cajaEnfocada]}>
        <TextInput
          {...props}
          style={[styles.entrada, style]}
```

```
          placeholderTextColor={colores.lapiz + '99'}
          secureTextEntry={esClave && !claveVisible}
          // Nunca autocorregir ni poner mayúscula inicial en correos y claves.
          autoCapitalize={props.autoCapitalize ?? 'none'}
          autoCorrect={false}
          accessibilityLabel={etiqueta}
          onFocus={(e) => {
            setEnfocado(true);
            props.onFocus?.(e);
          }}
          onBlur={(e) => {
            setEnfocado(false);
            props.onBlur?.(e);
          }}
        />
        {esClave && (
          <Pressable
            onPress={() => setClaveVisible((v) => !v)}
            hitSlop={8}
            accessibilityRole="button"
            accessibilityLabel={claveVisible ? 'Ocultar contraseña' : 'Mostrar contraseña'}>
            <Text style={styles.alternar}>{claveVisible ? 'Ocultar' : 'Mostrar'}</Text>
          </Pressable>
        )}
      </View>

      {/* accessibilityLiveRegion hace que Android lea el error en voz alta al aparecer. */}
      {error ? (
        <Text style={styles.error} accessibilityLiveRegion="polite">
          {error}
        </Text>
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  contenedor: {
    gap: espacio.s - 2,
  },
  etiqueta: {
    color: colores.grafito,
    fontFamily: fuentes.textoMedio,
    fontSize: 14,
  },
  caja: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: colores.hoja,
    borderWidth: 1.5,
    borderRadius: radio.control,
    paddingHorizontal: espacio.m,
    minHeight: 54,
```

```
  },
  cajaEnfocada: {
    // Al enfocar, una línea amarilla abajo refuerza dónde está el cursor.
    borderBottomColor: colores.senal,
    borderBottomWidth: 3,
  },
  entrada: {
    flex: 1,
    color: colores.grafito,
    fontFamily: fuentes.texto,
    fontSize: 16,
    paddingVertical: espacio.m - 2,
  },
  alternar: {
    color: colores.tinta,
    fontFamily: fuentes.textoFuerte,
    fontSize: 14,
    paddingLeft: espacio.s,
  },
  error: {
    color: colores.error,
    fontFamily: fuentes.textoMedio,
    fontSize: 13,
  },
});
```

El componente recibe todas las propiedades de un `TextInput` normal (con `...props`) y agrega las suyas: etiqueta, error y si es contraseña. Así se puede pasar `keyboardType`, `autoComplete` o cualquier otra propiedad sin modificar el componente.

10.4. Botón

Archivo **`src/components/boton.tsx`**

```
/**
 * Botón con tres variantes:
 *  - primario:   amarillo de seguridad. Una sola vez por pantalla.
 *  - secundario: contorno azul, para acciones alternativas.
 *  - enlace:     solo texto, para navegar ("¿Olvidaste tu contraseña?").
 *
 * El primario tiene una "base" oscura debajo y al presionarlo baja 4 px,
 * como una tecla física. La respuesta visual confirma que el toque se registró.
 */
import { ActivityIndicator, Pressable, StyleSheet, Text, View } from 'react-native';

import { colores, espacio, fuentes, radio } from '@/constants/tema';

type Props = {
  texto: string;
  onPress: () => void;
  variante?: 'primario' | 'secundario' | 'enlace';
  /** Muestra un indicador y bloquea el botón mientras se espera a Firebase. */
  cargando?: boolean;
  deshabilitado?: boolean;
```

```
};

const PROFUNDIDAD = 4;

export function Boton({
  texto,
  onPress,
  variante = 'primario',
  cargando = false,
  deshabilitado = false,
}: Props) {
  const inactivo = cargando || deshabilitado;

  if (variante === 'enlace') {
    return (
      <Pressable onPress={onPress} disabled={inactivo} hitSlop={8} accessibilityRole="link">
        {({ pressed }) => (
          <Text style={[styles.enlace, pressed && { opacity: 0.55 }]}>{texto}</Text>
        )}
      </Pressable>
    );
  }

  const esPrimario = variante === 'primario';

  return (
    <Pressable
      onPress={onPress}
      disabled={inactivo}
      accessibilityRole="button"
      accessibilityState={{ disabled: inactivo, busy: cargando }}
      style={styles.envoltura}>
      {({ pressed }) => (
        <>
          {/* Base que da la sensación de relieve (solo en el primario). */}
          {esPrimario && <View style={styles.base} />}
          <View
            style={[
              styles.cara,
              esPrimario ? styles.caraPrimaria : styles.caraSecundaria,
              // Al presionar, la cara baja hasta tocar la base.
              esPrimario && { transform: [{ translateY: pressed ? PROFUNDIDAD : 0 }] },
              !esPrimario && pressed && { backgroundColor: colores.tinta + '12' },
              inactivo && { opacity: 0.7 },
            ]}>
            {cargando ? (
              <ActivityIndicator color={esPrimario ? colores.tintaProfunda : colores.tinta} />
            ) : (
              <Text style={[styles.texto, !esPrimario && { color: colores.tinta }]}>{texto}</Text>
            )}
          </View>
        </>
      )}
```

```
    </Pressable>
  );
}

const styles = StyleSheet.create({
  envoltura: {
    marginTop: espacio.s,
    paddingBottom: PROFUNDIDAD,
  },
  base: {
    position: 'absolute',
    left: 0,
    right: 0,
    top: PROFUNDIDAD,
    bottom: 0,
    borderRadius: radio.control,
    backgroundColor: colores.tintaProfunda,
  },
  cara: {
    minHeight: 56,
    borderRadius: radio.control,
    alignItems: 'center',
    justifyContent: 'center',
    paddingHorizontal: espacio.l,
  },
  caraPrimaria: {
    backgroundColor: colores.senal,
  },
  caraSecundaria: {
    borderWidth: 1.5,
    borderColor: colores.tinta,
    backgroundColor: 'transparent',
  },
  texto: {
    color: colores.tintaProfunda,
    fontFamily: fuentes.textoFuerte,
    fontSize: 16,
  },
  enlace: {
    color: colores.tinta,
    fontFamily: fuentes.textoFuerte,
    fontSize: 15,
    textDecorationLine: 'underline',
    textDecorationColor: colores.senal,
  },
});
```

10.5. Aviso

Archivo **`src/components/aviso.tsx`**

```
/**
 * Mensaje de resultado (error o éxito) que aparece sobre el formulario.
```

```
 * La barra lateral de color permite distinguir el tipo sin leer el texto.
 */
import { StyleSheet, Text, View } from 'react-native';

import { colores, espacio, fuentes } from '@/constants/tema';

type Props = {
  tipo: 'error' | 'exito';
  mensaje: string;
};

export function Aviso({ tipo, mensaje }: Props) {
  const esError = tipo === 'error';
  return (
    <View
      // "alert" hace que el lector de pantalla anuncie el mensaje de inmediato.
      accessibilityRole="alert"
      style={[
        styles.caja,
        {
          backgroundColor: esError ? colores.errorFondo : colores.exitoFondo,
          borderLeftColor: esError ? colores.error : colores.exito,
        },
      ]}>
      <Text style={[styles.texto, { color: esError ? colores.error : colores.exito }]}>
        {mensaje}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  caja: {
    borderLeftWidth: 4,
    borderRadius: 8,
    paddingVertical: espacio.m - 4,
    paddingHorizontal: espacio.m,
  },
  texto: {
    fontFamily: fuentes.textoMedio,
    fontSize: 14,
    lineHeight: 20,
  },
});
```

11. Navegación protegida

11.1. Layout raíz

El layout raíz es lo primero que se ejecuta. Carga las fuentes, crea el proveedor de sesión y define qué rutas existen según haya o no un usuario conectado.

Archivo **`src/app/_layout.tsx`**

```tsx
/**
 * Layout raíz: es lo primero que se ejecuta al abrir la app.
 *
 * Responsabilidades:
 *   1. Cargar las fuentes.
 *   2. Envolver la app en el ProveedorSesion (para que todas las pantallas
 *      conozcan al usuario).
 *   3. Decidir qué pantallas existen según haya o no sesión (Stack.Protected).
 *   4. Mantener visible la pantalla de carga (splash) hasta tener todo listo.
 */
import { BigShoulders_800ExtraBold } from '@expo-google-fonts/big-shoulders';
import {
  SpaceGrotesk_400Regular,
  SpaceGrotesk_500Medium,
  SpaceGrotesk_700Bold,
  useFonts,
} from '@expo-google-fonts/space-grotesk';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { useEffect } from 'react';

import { colores } from '@/constants/tema';
import { ProveedorSesion, useSesion } from '@/context/sesion';

// Se ejecuta al importar el módulo: evita que el splash se oculte solo.
SplashScreen.preventAutoHideAsync();

export default function LayoutRaiz() {
  // useFonts de cualquier paquete @expo-google-fonts sirve para cargar fuentes
  // de distintas familias a la vez.
  const [fuentesListas] = useFonts({
    BigShoulders_800ExtraBold,
    SpaceGrotesk_400Regular,
    SpaceGrotesk_500Medium,
    SpaceGrotesk_700Bold,
  });

  return (
    <ProveedorSesion>
      <Navegacion fuentesListas={fuentesListas} />
    </ProveedorSesion>
  );
}

/**
 * Está separado de LayoutRaiz porque useSesion solo funciona DENTRO del
 * ProveedorSesion, y LayoutRaiz es justamente quien lo crea.
 */
function Navegacion({ fuentesListas }: { fuentesListas: boolean }) {
  const { usuario, cargando } = useSesion();
  const listo = fuentesListas && !cargando;
```

```
  useEffect(() => {
    // Cuando hay fuentes y Firebase ya respondió, quitamos el splash.
    if (listo) SplashScreen.hideAsync();
  }, [listo]);

  // Mientras tanto no dibujamos nada: el splash sigue cubriendo la pantalla.
  if (!listo) return null;

  return (
    <Stack
      screenOptions={{
        headerShown: false, // cada pantalla dibuja su propio encabezado
        contentStyle: { backgroundColor: colores.tinta },
        animation: 'slide_from_right',
      }}>
      {/*
        Stack.Protected: si `guard` es false, esas rutas "no existen".
        Si alguien está en una ruta que deja de existir (por ejemplo, cierra
        sesión estando en la bienvenida), Expo Router lo lleva a la primera
        ruta disponible. Así no hace falta escribir redirecciones a mano.
      */}
      <Stack.Protected guard={!!usuario}>
        <Stack.Screen name="(app)" />
      </Stack.Protected>

      <Stack.Protected guard={!usuario}>
        <Stack.Screen name="login" />
        <Stack.Screen name="registro" />
        <Stack.Screen name="recuperar" />
      </Stack.Protected>
    </Stack>
  );
}
```

`Stack.Protected` hace el trabajo que antes requería redirecciones manuales. Cuando `guard` es falso, esas rutas dejan de existir. Si el usuario está en la bienvenida y cierra sesión, la ruta desaparece y Expo Router lo lleva a la primera ruta disponible, que es el login. Lo mismo ocurre al revés tras iniciar sesión.

El componente `Navegacion` está separado de `LayoutRaiz` por que `useSesion` solo funciona dentro de `ProveedorSesion`, y `LayoutRaiz` es justamente quien crea ese proveedor.

11.2. Layout del grupo protegido

Archivo **`src/app/(app)/_layout.tsx`**



12. Pantallas

12.1. Inicio de sesión

Archivo **`src/app/login.tsx`**

```tsx
/**
 * Pantalla de inicio de sesión.
 */
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, Text, View } from 'react-native';

import { Aviso } from '@/components/aviso';
import { Boton } from '@/components/boton';
import { CampoTexto } from '@/components/campo-texto';
import { PantallaHoja } from '@/components/pantalla-hoja';
import { colores, espacio, fuentes } from '@/constants/tema';
import { useSesion } from '@/context/sesion';
import { mensajeDeError } from '@/lib/errores-firebase';
import { validarCorreo } from '@/lib/validaciones';

export default function PantallaLogin() {
  const { iniciarSesion } = useSesion();

  // Estado del formulario: un useState por campo es lo más claro para empezar.
  const [correo, setCorreo] = useState('');
  const [clave, setClave] = useState('');
  const [errores, setErrores] = useState<{ correo?: string | null; clave?: string | null }>({});
  const [errorGeneral, setErrorGeneral] = useState<string | null>(null);
  const [enviando, setEnviando] = useState(false);

  async function manejarIngreso() {
    // 1. Validamos en el teléfono antes de llamar a Firebase.
    const nuevosErrores = {
      correo: validarCorreo(correo),
      clave: clave ? null : 'Escribe tu contraseña.',
```

```
    };
    setErrores(nuevosErrores);
    setErrorGeneral(null);
    if (nuevosErrores.correo || nuevosErrores.clave) return;

    // 2. Llamamos a Firebase. Si funciona, el layout raíz cambia de pantalla solo.
    setEnviando(true);
    try {
      await iniciarSesion(correo, clave);
    } catch (error) {
      setErrorGeneral(mensajeDeError(error));
      // finally no sirve aquí para todo: si el ingreso funciona, esta pantalla
      // se desmonta y no debemos actualizar su estado.
      setEnviando(false);
    }
  }

  return (
    <PantallaHoja
      antetitulo="ESFOT, Escuela Politécnica Nacional"
      titulo={'Inicia\nsesión'}
      bajada="Ingresa con el correo y la contraseña de tu cuenta.">
      {errorGeneral ? <Aviso tipo="error" mensaje={errorGeneral} /> : null}

      <CampoTexto
        etiqueta="Correo"
        placeholder="nombre.apellido@epn.edu.ec"
        value={correo}
        onChangeText={setCorreo}
        error={errores.correo}
        keyboardType="email-address"
        textContentType="emailAddress" // permite autocompletar en iOS
        autoComplete="email" // y en Android
        returnKeyType="next"
      />

      <CampoTexto
        etiqueta="Contraseña"
        value={clave}
        onChangeText={setClave}
        error={errores.clave}
        esClave
        textContentType="password"
        autoComplete="current-password"
        returnKeyType="go"
        onSubmitEditing={manejarIngreso} // "Enter" en el teclado también envía
      />

      <View style={styles.olvido}>
        <Boton
          variante="enlace"
          texto="¿Olvidaste tu contraseña?"
          // Pasamos el correo escrito para no obligar a escribirlo otra vez.
```

```
          onPress={() => router.push({ pathname: '/recuperar', params: { correo } })}
        />
      </View>

      <Boton texto="Iniciar sesión" onPress={manejarIngreso} cargando={enviando} />

      <View style={styles.pie}>
        <Text style={styles.pieTexto}>¿Aún no tienes cuenta?</Text>
        <Boton variante="enlace" texto="Crea una" onPress={() => router.push('/registro')} />
      </View>
    </PantallaHoja>
  );
}

const styles = StyleSheet.create({
  olvido: {
    alignItems: 'flex-end',
  },
  pie: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    gap: espacio.s - 2,
    marginTop: espacio.m,
  },
  pieTexto: {
    color: colores.lapiz,
    fontFamily: fuentes.texto,
    fontSize: 15,
  },
});
```

El enlace "¿Olvidaste tu contraseña?" envía el correo ya escrito como parámetro de ruta, para que el estudiante no tenga que escribirlo dos veces.

12.2. Registro

Archivo **`src/app/registro.tsx`**

```
/**
 * Pantalla de registro de una cuenta nueva.
 *
 * Ojo: este registro es abierto. Cualquier persona con un correo puede crear
 * una cuenta. El texto "estudiante de la ESFOT" de la bienvenida no verifica
 * nada; para restringir de verdad habría que crear las cuentas desde la consola
 * o validar el dominio en el servidor.
 */
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, Text, View } from 'react-native';

import { Aviso } from '@/components/aviso';
import { Boton } from '@/components/boton';
import { CampoTexto } from '@/components/campo-texto';
```

```tsx
import { PantallaHoja } from '@/components/pantalla-hoja';
import { colores, espacio, fuentes } from '@/constants/tema';
import { useSesion } from '@/context/sesion';
import { mensajeDeError } from '@/lib/errores-firebase';
import {
  LONGITUD_MINIMA_CLAVE,
  validarClave,
  validarConfirmacion,
  validarCorreo,
  validarNombre,
} from '@/lib/validaciones';

type Errores = Partial<Record<'nombre' | 'correo' | 'clave' | 'confirmacion', string | null>>;

export default function PantallaRegistro() {
  const { registrarse } = useSesion();

  const [nombre, setNombre] = useState('');
  const [correo, setCorreo] = useState('');
  const [clave, setClave] = useState('');
  const [confirmacion, setConfirmacion] = useState('');
  const [errores, setErrores] = useState<Errores>({});
  const [errorGeneral, setErrorGeneral] = useState<string | null>(null);
  const [enviando, setEnviando] = useState(false);

  async function manejarRegistro() {
    const nuevosErrores: Errores = {
      nombre: validarNombre(nombre),
      correo: validarCorreo(correo),
      clave: validarClave(clave),
      confirmacion: validarConfirmacion(clave, confirmacion),
    };
    setErrores(nuevosErrores);
    setErrorGeneral(null);
    // Object.values + some: ¿hay al menos un error?
    if (Object.values(nuevosErrores).some(Boolean)) return;

    setEnviando(true);
    try {
      await registrarse(nombre, correo, clave);
      // Al crear la cuenta Firebase inicia sesión y el layout muestra la bienvenida.
    } catch (error) {
      setErrorGeneral(mensajeDeError(error));
      setEnviando(false);
    }
  }

  return (
    <PantallaHoja
      conVolver
      titulo={'Crea tu\ncuenta'}
      bajada="Con esta cuenta entrarás a la app desde cualquier teléfono.">
      {errorGeneral ? <Aviso tipo="error" mensaje={errorGeneral} /> : null}
```

```
      <CampoTexto
        etiqueta="Nombre"
        placeholder="Como quieres que te saludemos"
        value={nombre}
        onChangeText={setNombre}
        error={errores.nombre}
        autoCapitalize="words"
        textContentType="name"
        autoComplete="name"
      />
      <CampoTexto
        etiqueta="Correo"
        placeholder="nombre.apellido@epn.edu.ec"
        value={correo}
        onChangeText={setCorreo}
        error={errores.correo}
        keyboardType="email-address"
        textContentType="emailAddress"
        autoComplete="email"
      />
      <CampoTexto
        etiqueta="Contraseña"
        placeholder={`Mínimo ${LONGITUD_MINIMA_CLAVE} caracteres`}
        value={clave}
        onChangeText={setClave}
        error={errores.clave}
        esClave
        // "newPassword" hace que iOS sugiera una contraseña segura.
        textContentType="newPassword"
        autoComplete="new-password"
      />
      <CampoTexto
        etiqueta="Repite la contraseña"
        value={confirmacion}
        onChangeText={setConfirmacion}
        error={errores.confirmacion}
        esClave
        textContentType="newPassword"
        autoComplete="new-password"
        onSubmitEditing={manejarRegistro}
      />

      <Boton texto="Crear cuenta" onPress={manejarRegistro} cargando={enviando} />

      <View style={styles.pie}>
        <Text style={styles.pieTexto}>¿Ya tienes cuenta?</Text>
        <Boton variante="enlace" texto="Inicia sesión" onPress={() => router.back()} />
      </View>
    </PantallaHoja>
  );
}
```

```
const styles = StyleSheet.create({
  pie: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    gap: espacio.s - 2,
    marginTop: espacio.m,
  },
  pieTexto: {
    color: colores.lapiz,
    fontFamily: fuentes.texto,
    fontSize: 15,
  },
});
```

Las propiedades `textContentType="newPassword"` y `autoComplete="new-password"` le indican al sistema que es una contraseña nueva. iOS y Android pueden sugerir una contraseña segura y guardarla en el gestor de contraseñas del teléfono.

12.3. Recuperar contraseña

Archivo **`src/app/recuperar.tsx`**

```
/**
 * Recuperar contraseña.
 *
 * Flujo completo:
 *   1. El estudiante escribe su correo y toca "Enviar enlace".
 *   2. Firebase envía un correo (plantilla editable en la consola:
 *      Authentication > Templates > Password reset).
 *   3. El enlace abre una página de Firebase donde se escribe la contraseña nueva.
 *   4. El estudiante vuelve a la app e inicia sesión con esa contraseña.
 */
import { router, useLocalSearchParams } from 'expo-router';
import { useState } from 'react';

import { Aviso } from '@/components/aviso';
import { Boton } from '@/components/boton';
import { CampoTexto } from '@/components/campo-texto';
import { PantallaHoja } from '@/components/pantalla-hoja';
import { useSesion } from '@/context/sesion';
import { mensajeDeError } from '@/lib/errores-firebase';
import { validarCorreo } from '@/lib/validaciones';

export default function PantallaRecuperar() {
  const { enviarCorreoRecuperacion } = useSesion();
  // Recibimos el correo que el usuario ya había escrito en el login (si lo hizo).
  const params = useLocalSearchParams<{ correo?: string }>();

  const [correo, setCorreo] = useState(params.correo ?? '');
  const [errorCampo, setErrorCampo] = useState<string | null>(null);
  const [errorGeneral, setErrorGeneral] = useState<string | null>(null);
  const [enviado, setEnviado] = useState(false);
  const [enviando, setEnviando] = useState(false);
```

```
  async function manejarEnvio() {
    const error = validarCorreo(correo);
    setErrorCampo(error);
    setErrorGeneral(null);
    if (error) return;

    setEnviando(true);
    try {
      await enviarCorreoRecuperacion(correo);
      setEnviado(true);
    } catch (e) {
      setErrorGeneral(mensajeDeError(e));
    } finally {
      // Aquí sí usamos finally: la pantalla sigue montada en ambos casos.
      setEnviando(false);
    }
  }

  return (
    <PantallaHoja
      conVolver
      titulo={'Recupera\ntu acceso'}
      bajada="Te enviaremos un enlace para que crees una contraseña nueva.">
      {/*
        Mensaje neutro a propósito: no confirmamos si el correo existe.
        Decir "ese correo no está registrado" permitiría a un atacante
        averiguar qué correos tienen cuenta (enumeración de usuarios).
      */}
      {enviado ? (
        <Aviso
          tipo="exito"
          mensaje={`Si ${correo.trim()} tiene una cuenta, en unos minutos recibirás el enlace.
Revisa también la carpeta de spam.`}
        />
      ) : null}
      {errorGeneral ? <Aviso tipo="error" mensaje={errorGeneral} /> : null}

      <CampoTexto
        etiqueta="Correo de tu cuenta"
        placeholder="nombre.apellido@epn.edu.ec"
        value={correo}
        onChangeText={(texto) => {
          setCorreo(texto);
          setEnviado(false); // si cambia el correo, el aviso anterior ya no aplica
        }}
        error={errorCampo}
        keyboardType="email-address"
        textContentType="emailAddress"
        autoComplete="email"
        onSubmitEditing={manejarEnvio}
      />
```

```
      {enviado ? (
        <>
          <Boton texto="Volver a iniciar sesión" onPress={() => router.back()} />
          <Boton variante="secundario" texto="Enviar otra vez" onPress={manejarEnvio}
cargando={enviando} />
        </>
      ) : (
        <Boton texto="Enviar enlace" onPress={manejarEnvio} cargando={enviando} />
      )}
    </PantallaHoja>
  );
}
```

El mensaje de éxito es neutro a propósito: "si el correo tiene una cuenta, recibirás el enlace". Si dijéramos "ese correo no está registrado", cualquiera podría usar la pantalla para averiguar qué correos tienen cuenta.

El flujo completo para el estudiante es: escribe su correo, recibe el mensaje, abre el enlace, escribe la contraseña nueva en la página de Firebase, vuelve a la app e inicia sesión con esa contraseña.

12.4. Bienvenida

Archivo **`src/app/(app)/index.tsx`**

```tsx
/**
 * Pantalla de bienvenida (solo con sesión iniciada).
 */
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, Text, View } from 'react-native';
import Animated, { FadeInDown } from 'react-native-reanimated';

import { Boton } from '@/components/boton';
import { PantallaHoja } from '@/components/pantalla-hoja';
import { colores, espacio, fuentes, radio } from '@/constants/tema';
import { useSesion } from '@/context/sesion';

/** Convierte la fecha que entrega Firebase (texto) a "27 de octubre de 2026". */
function formatearFecha(fecha?: string): string {
  if (!fecha) return 'Sin registro';
  return new Date(fecha).toLocaleDateString('es-EC', {
    day: 'numeric',
    month: 'long',
    year: 'numeric',
  });
}

export default function PantallaBienvenida() {
  const { usuario, nombre: nombrePerfil, cerrarSesion } = useSesion();
  const [saliendo, setSaliendo] = useState(false);

  // Gracias a Stack.Protected esta pantalla nunca se muestra sin usuario,
  // pero TypeScript no lo sabe; por eso el operador ?.
  // Para el saludo basta el primer nombre; la ficha muestra el nombre completo.
```

```
  const nombre =
    nombrePerfil?.split(' ')[0] || usuario?.email?.split('@')[0] || 'estudiante';

  async function manejarSalida() {
    setSaliendo(true);
    await cerrarSesion();
    // No navegamos: al quedar usuario = null, Stack.Protected lleva al login.
  }

  return (
    <PantallaHoja
      titulo={'Bienvenido\na la APP,\nestudiante\nde la ESFOT'}
      extraEncabezado={
        // La única animación de la app: confirma que el ingreso funcionó.
        // Reanimated respeta la opción "reducir movimiento" del sistema.
        <Animated.Text entering={FadeInDown.duration(1000).delay(150)} style={styles.saludo}>
          Hola, {nombre}.
        </Animated.Text>
      }>
      {/* Ficha con los datos de la cuenta que guarda Firebase. */}
      <View style={styles.ficha}>
        <Fila etiqueta="Nombre" valor={nombrePerfil || 'Sin nombre'} />
        <Fila etiqueta="Correo" valor={usuario?.email ?? ''} />
        <Fila etiqueta="Cuenta creada" valor={formatearFecha(usuario?.metadata.creationTime)} />
        <Fila
          etiqueta="Último ingreso"
          valor={formatearFecha(usuario?.metadata.lastSignInTime)}
          ultima
        />
      </View>

      <Boton texto="Cambiar contraseña" onPress={() => router.push('/cambiar-clave')} />
      <Boton variante="secundario" texto="Cerrar sesión" onPress={manejarSalida} cargando={saliendo} />
    </PantallaHoja>
  );
}

/** Una fila etiqueta-valor de la ficha. Componente local: solo se usa aquí. */
function Fila({ etiqueta, valor, ultima = false }: { etiqueta: string; valor: string; ultima?: boolean }) {
  return (
    <View style={[styles.fila, !ultima && styles.filaSeparada]}>
      <Text style={styles.filaEtiqueta}>{etiqueta}</Text>
      <Text style={styles.filaValor} numberOfLines={1} ellipsizeMode="middle">
        {valor}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  saludo: {
    color: colores.senal,
    fontFamily: fuentes.textoMedio,
```

```
    fontSize: 18,
    marginTop: espacio.m,
  },
  ficha: {
    backgroundColor: colores.hoja,
    borderRadius: radio.control + 4,
    paddingHorizontal: espacio.m,
    borderWidth: 1,
    borderColor: colores.borde,
  },
  fila: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    gap: espacio.m,
    paddingVertical: espacio.m - 2,
  },
  filaSeparada: {
    borderBottomWidth: StyleSheet.hairlineWidth,
    borderBottomColor: colores.borde,
  },
  filaEtiqueta: {
    color: colores.lapiz,
    fontFamily: fuentes.texto,
    fontSize: 14,
  },
  filaValor: {
    flexShrink: 1,
    color: colores.grafito,
    fontFamily: fuentes.textoMedio,
    fontSize: 15,
    textAlign: 'right',
  },
});
```

La pantalla muestra el mensaje pedido, saluda por el primer nombre y presenta una ficha con los datos que Firebase guarda de cada cuenta: nombre, correo, fecha de creación y último ingreso. Es la única pantalla con animación: el saludo aparece con un desplazamiento corto que confirma que el ingreso funcionó. Para eso se usa react-native-reanimated como librería de animaciones, no es necesario instalarla porque ya viene integrada cuando se crea un proyecto con EXPO SDK 57.

12.5. Cambiar contraseña

Archivo **`src/app/(app)/cambiar-clave.tsx`**

```
/**
 * Cambiar contraseña (solo con sesión iniciada).
 *
 * Pedimos la contraseña actual por dos razones:
 *   1. Firebase exige un inicio de sesión reciente para cambiar la clave
 *      (si no, lanza auth/requires-recent-login). Con la clave actual
 *      reautenticamos al usuario en el momento.
```

```
 *   2. Si alguien toma un teléfono desbloqueado, no puede cambiar la clave
 *      sin conocer la actual.
 */
import { router } from 'expo-router';
import { useState } from 'react';

import { Aviso } from '@/components/aviso';
import { Boton } from '@/components/boton';
import { CampoTexto } from '@/components/campo-texto';
import { PantallaHoja } from '@/components/pantalla-hoja';
import { useSesion } from '@/context/sesion';
import { mensajeDeError } from '@/lib/errores-firebase';
import { LONGITUD_MINIMA_CLAVE, validarClave, validarConfirmacion } from '@/lib/validaciones';

type Errores = Partial<Record<'actual' | 'nueva' | 'confirmacion', string | null>>;

export default function PantallaCambiarClave() {
  const { cambiarClave } = useSesion();

  const [actual, setActual] = useState('');
  const [nueva, setNueva] = useState('');
  const [confirmacion, setConfirmacion] = useState('');
  const [errores, setErrores] = useState<Errores>({});
  const [errorGeneral, setErrorGeneral] = useState<string | null>(null);
  const [listo, setListo] = useState(false);
  const [enviando, setEnviando] = useState(false);

  async function manejarCambio() {
    const nuevosErrores: Errores = {
      actual: actual ? null : 'Escribe tu contraseña actual.',
      nueva:
        validarClave(nueva) ??
        (nueva === actual ? 'La nueva contraseña debe ser distinta de la actual.' : null),
      confirmacion: validarConfirmacion(nueva, confirmacion),
    };
    setErrores(nuevosErrores);
    setErrorGeneral(null);
    if (Object.values(nuevosErrores).some(Boolean)) return;

    setEnviando(true);
    try {
      await cambiarClave(actual, nueva);
      setListo(true);
      // Limpiamos los campos: no conviene dejar contraseñas en memoria visible.
      setActual('');
      setNueva('');
      setConfirmacion('');
    } catch (error) {
      setErrorGeneral(
        // En esta pantalla, "credencial inválida" significa clave actual incorrecta.
        mensajeDeError(error, {
          'auth/invalid-credential': 'La contraseña actual no es correcta.',
          'auth/wrong-password': 'La contraseña actual no es correcta.',
```

```
      }),
    );
  } finally {
    setEnviando(false);
  }
}

return (
  <PantallaHoja
    conVolver
    titulo={'Cambia tu\ncontraseña'}
    bajada={`Confirma la actual y elige una nueva de al menos ${LONGITUD_MINIMA_CLAVE} caracteres.`}>
    {listo ? (
      <Aviso
        tipo="exito"
        mensaje="Contraseña actualizada. La próxima vez que inicies sesión usa la nueva."
      />
    ) : null}
    {errorGeneral ? <Aviso tipo="error" mensaje={errorGeneral} /> : null}

    <CampoTexto
      etiqueta="Contraseña actual"
      value={actual}
      onChangeText={setActual}
      error={errores.actual}
      esClave
      textContentType="password"
      autoComplete="current-password"
    />
    <CampoTexto
      etiqueta="Contraseña nueva"
      value={nueva}
      onChangeText={setNueva}
      error={errores.nueva}
      esClave
      textContentType="newPassword"
      autoComplete="new-password"
    />
    <CampoTexto
      etiqueta="Repite la contraseña nueva"
      value={confirmacion}
      onChangeText={setConfirmacion}
      error={errores.confirmacion}
      esClave
      textContentType="newPassword"
      autoComplete="new-password"
      onSubmitEditing={manejarCambio}
    />

    {listo ? (
      <Boton texto="Volver a la bienvenida" onPress={() => router.back()} />
    ) : (
      <Boton texto="Guardar contraseña" onPress={manejarCambio} cargando={enviando} />
```

```
      )}
    </PantallaHoja>
  );
}
```

Aquí se ve el uso de los reemplazos de `mensajeDeError`. En el login, `auth/invalid-credential` significa "correo o contraseña incorrectos"; en esta pantalla significa "la contraseña actual no es correcta". El mismo código de error tiene distinto significado según el contexto, y el mensaje debe reflejarlo.

13. Ejecución y pruebas

13.1. Ejecutar la app

```
npx expo start
```

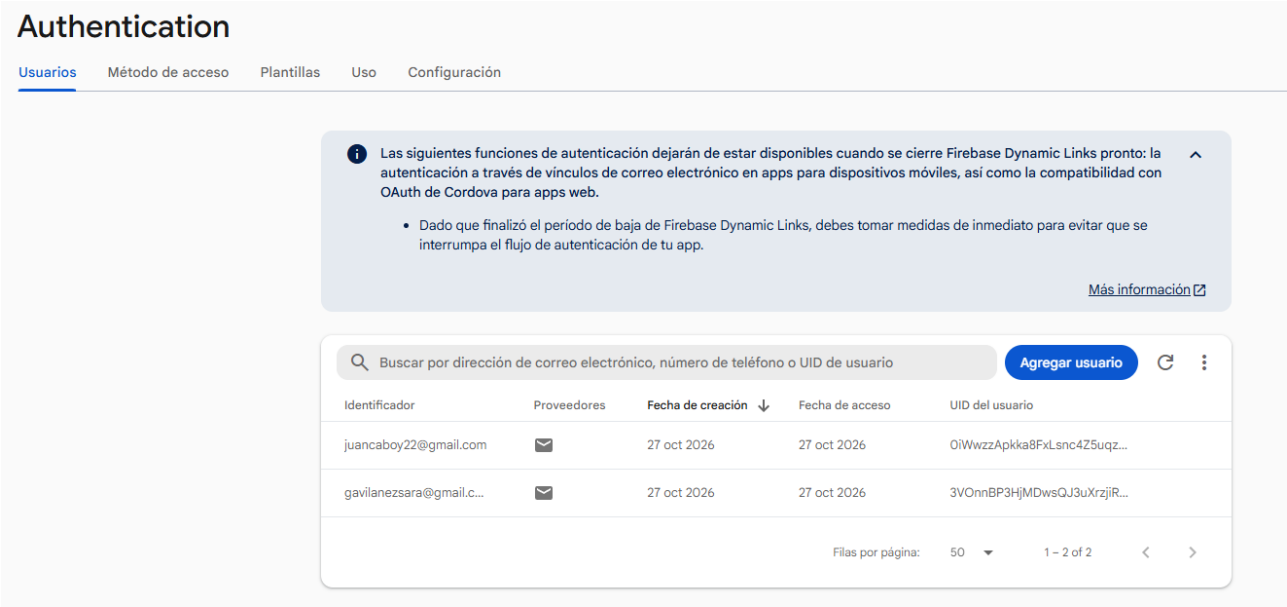
Se escanea el código QR con la cámara (iOS) o con Expo Go (Android).

13.2. Casos de prueba

Se recomienda que cada estudiante complete esta lista en su teléfono y anote el resultado obtenido.

Caso	**Pasos**	**Resultado esperado**
Campos vacíos	Tocar "Iniciar sesión" sin escribir nada.	Mensajes en rojo debajo de cada campo, sin llamar a Firebase.
Registro correcto	Crear cuenta con nombre, correo y contraseña de 8 caracteres.	Pasa directo a la bienvenida y saluda por el nombre.
Correo repetido	Registrar otra vez el mismo correo.	"Ya existe una cuenta con ese correo..."
Contraseñas distintas	En registro, escribir confirmación diferente.	"Las contraseñas no coinciden."
Credenciales incorrectas	Iniciar sesión con una contraseña equivocada.	Aviso: "El correo o la contraseña no coinciden..."
Sesión persistente	Iniciar sesión, cerrar la app por completo y abrirla.	Abre directo en la bienvenida.
Recuperación	Pedir enlace, abrir el correo, cambiar la contraseña e ingresar con ella.	Llega el correo en español y la nueva contraseña funciona.
Cambio con clave actual errónea	En cambiar contraseña, escribir mal la actual.	"La contraseña actual no es correcta."
Cambio correcto	Cambiar la contraseña, cerrar sesión e ingresar con la nueva.	Aviso de éxito y el ingreso funciona solo con la nueva.
Cerrar sesión	Tocar "Cerrar sesión" en la bienvenida.	Vuelve al login y el botón atrás no regresa a la bienvenida.

En la consola de Firebase, pestaña **Users**, se pueden ver las cuentas creadas, su fecha de creación y su último ingreso.



14. Errores frecuentes

Síntoma	Causa probable	Solución
`auth/operation-not-allowed`	No se habilitó Correo/contraseña.	Sección 3.3.
`auth/invalid-api-key` o `api-key-not-valid`	Valores mal copiados o falta `.env.local`.	Revisar el archivo y reiniciar con `npx expo start -c`.
TypeScript marca `getReactNativePersistence`	Falta el ajuste de `paths`.	Sección 6.1.
La sesión se pierde al cerrar la app	Se usó `getAuth` en lugar de `initializeAuth` con persistencia.	Revisar `src/config/firebase.ts`.
`auth/requires-recent-login`	Se llamó a `updatePassword` sin reautenticar.	Usar `reauthenticateWithCredential` antes.
El correo de recuperación no llega	Filtro de spam, o el correo no tiene cuenta.	Revisar spam; verificar la cuenta en la pestaña Users.
Expo Go dice que el proyecto es incompatible	Expo Go y el proyecto usan SDK distintos.	Actualizar Expo Go o crear el proyecto con el SDK que soporta.

Reto

1) **Ha surgido un nueva HU a su Sprint Backlog y ahora le piden añadir la funcionalidad de verificación de correo.** Después del registro de cuenta, se debe enviar un correo de verificación con `sendEmailVerification` de FireBase, SOLO si la cuenta se ha verificado, se puede pasar a la pantalla de bienvenido.
2) Consultar en la documentacion oficial de react native reanimated:

   https://docs.swmansion.com/react-native-reanimated/

   La animación **Shake en error:** cuando el login falla, el campo o la hoja se sacude e implementarla.

   Nota: La parte básica del taller vale 10 puntos, cada reto 5 puntos(por hoy no hay defensa de código, pero no se acostumbren)

   Entregable:

   Adjuntar Link de GitHub de su app:

   Subir al aula virtual el pdf y demostrar en clase al docente la culminación.